The Computation Graph & Backprop, Worked Out by Hand

Reverse-mode autodiff on a tiny graph, every local derivative shown

A complete numerical walk-through of how PyTorch turns a handful of tensor operations into a directed graph, runs a *forward* pass that stores values, and then runs a single *backward* pass that hands back the gradient of the loss with respect to *every* leaf at once. We take the smallest graph that still has a multiply, an add, and a `relu` — $f = (a \cdot b) + \mathrm{relu}(c)$ — and compute every local derivative and every chain-rule product by hand. Each number below was produced by the NumPy/PyTorch reference program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It fixes $a = 2.0,\ b = -3.0,\ c = 4.0$, draws the graph, labels every node and edge, and then performs reverse-mode automatic differentiation step by step: forward to get $f = -2.0$, then backward to get $\partial L/\partial a = -3$, $\partial L/\partial b = 2$, $\partial L/\partial c = 1$ — confirming each against `torch.autograd`.

How to read this. Each idea gets its own worked step with exact numbers, and the coloured boxes isolate the single mechanism in play (one multiply node, the dead-ReLU branch, gradient accumulation). The graph picture in Step 1 is the map; every later step is one edge of that map being multiplied through.

Concepts covered, in order: what the graph is (nodes, edges, leaves) · the forward pass that stores values · local derivatives of $\times$, $+$, and `relu` · the chain rule in reverse topological order · why one backward pass yields all leaf grads (reverse-mode efficiency) · the dead-ReLU case $c < 0$ · fan-out $y = x \cdot x$ where gradients *add* at a fork · why `.grad` accumulates and must be zeroed between steps.

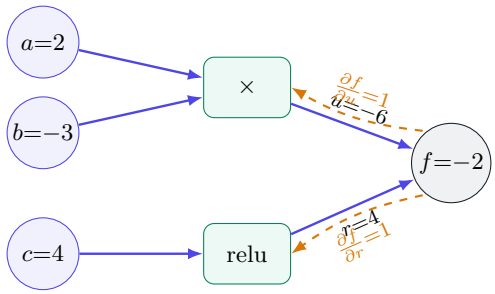
1 The setup: a graph small enough to differentiate by hand

When you do arithmetic on PyTorch tensors that carry `requires_grad=True`, the library silently records a directed acyclic graph (DAG): leaves are the input tensors, interior nodes are the operations, and edges carry intermediate values forward. Calling `.backward()` walks that graph in reverse, applying the chain rule at each node. To see the whole machine on one page we pick the smallest graph that still exercises a multiply, an add, and a nonlinearity:

$$\boxed{f \;=\; (a \cdot b) \;+\; \mathrm{relu}(c)}, \qquad a = 2.0, \quad b = -3.0, \quad c = 4.0, \qquad L = f.$$

Role	Symbol	Value here
Leaf input (tracked)	a	2.0
Leaf input (tracked)	b	-3.0
Leaf input (tracked)	c	4.0
Interior: product node	$u = a \cdot b$	-6.0
Interior: relu node	$r = \mathrm{relu}(c)$	4.0
Output / loss	$f = u + r = L$	-2.0

The graph, drawn. Three leaves on the left feed two operations, which feed the sum f on the right. Solid indigo arrows are the *forward* edges (values flowing left to right); dashed amber arrows are the *backward* edges (gradients flowing right to left).



Intuition

A computation graph is not a flowchart you have to interpret — it is a literal record of *which value was made from which*, kept so the chain rule can be applied mechanically. Forward, each node multiplies its inputs (or adds, or clips) and *stores* the result. Backward, each node already knows its own local derivative, so differentiating the whole expression is just multiplying those stored locals together along every path. No symbolic algebra, no finite differences — just bookkeeping.

2 Step 1 — The forward pass: compute and store every value

Reverse-mode autodiff cannot run backward until the forward pass has filled in the intermediate values, because the local derivatives depend on them (the derivative of a product needs the *other*

factor). So we evaluate the graph left to right and record each node:

$$u = a \cdot b = (2.0) \cdot (-3.0) = -6.0,$$
$$r = \text{relu}(c) = \max(0,\, 4.0) = 4.0,$$
$$f = u + r = (-6.0) + (4.0) = -2.0 \;=\; L.$$

These three stored numbers — $u = -6.0$, $r = 4.0$, $f = -2.0$ — are exactly what PyTorch keeps alive in the graph after the forward call. They are the inputs the backward pass will read.

Mini-example — this operation in isolation

A single multiply node in isolation. Let $u = a \cdot b$. Its two local derivatives are the *opposite* operands:

$$\frac{\partial u}{\partial a} = b, \qquad \frac{\partial u}{\partial b} = a.$$

At $a = 2$, $b = -3$ this gives $\partial u/\partial a = -3$ and $\partial u/\partial b = 2$. Notice the node must have *remembered* a and b from the forward pass to report these — that is why autodiff stores activations, and why a long forward pass costs memory.

3 Step 2 — Local derivatives: one per edge

Every edge of the graph carries a *local* derivative: how much the node's output moves when its input moves, holding the other inputs fixed. These are textbook one-liners.

The add node $f = u + r$. A sum passes gradient through untouched:

$$\frac{\partial f}{\partial u} = 1, \qquad \frac{\partial f}{\partial r} = 1.$$

The multiply node $u = a \cdot b$. Each partial is the other factor:

$$\frac{\partial u}{\partial a} = b = -3, \qquad \frac{\partial u}{\partial b} = a = 2.$$

The relu node $r = \max(0, c)$. Its derivative is the step function — 1 where the input is positive, 0 where negative:

$$\frac{\partial r}{\partial c} \;=\; \begin{cases} 1 & c > 0, \\ 0 & c < 0. \end{cases} \qquad \text{Here } c = 4 > 0 \Rightarrow \frac{\partial r}{\partial c} = 1.$$

Edge	Node	Local derivative	Value at our point
$f \leftarrow u$	add	$\partial f/\partial u = 1$	1
$f \leftarrow r$	add	$\partial f/\partial r = 1$	1
$u \leftarrow a$	multiply	$\partial u/\partial a = b$	-3
$u \leftarrow b$	multiply	$\partial u/\partial b = a$	2
$r \leftarrow c$	relu	$\partial r/\partial c = [c > 0]$	1

These five numbers are the entire local-derivative inventory of the graph. The backward pass does nothing more than multiply them along paths.

4 Step 3 — The backward pass: chain rule in reverse order

Reverse-mode AD seeds the output with $\partial L/\partial f = 1$ (the loss with respect to itself), then walks the nodes in *reverse topological order* — output first, leaves last — multiplying each incoming gradient by the node's local derivative. The running quantity attached to a node is its *adjoint* $\bar{v} = \partial L/\partial v$.

Seed. $\bar{f} = \dfrac{\partial L}{\partial f} = 1.$

Through the add node. Push $\bar{f}$ down both edges, multiplying by the local 1s:

$$\bar{u} = \bar{f} \cdot \frac{\partial f}{\partial u} = 1 \cdot 1 = 1, \qquad \bar{r} = \bar{f} \cdot \frac{\partial f}{\partial r} = 1 \cdot 1 = 1.$$

Through the multiply node (uses $\bar{u} = 1$):

$$\frac{\partial L}{\partial a} = \bar{u} \cdot \frac{\partial u}{\partial a} = 1 \cdot (-3) = \boxed{-3}, \qquad \frac{\partial L}{\partial b} = \bar{u} \cdot \frac{\partial u}{\partial b} = 1 \cdot 2 = \boxed{2}.$$

Through the relu node (uses $\bar{r} = 1$):

$$\frac{\partial L}{\partial c} = \bar{r} \cdot \frac{\partial r}{\partial c} = 1 \cdot 1 = \boxed{1}.$$

Every leaf gradient fell out of a *single* sweep. We never re-evaluated f; we only multiplied stored locals. PyTorch agrees exactly: `a.grad`$= -3$, `b.grad`$= 2$, `c.grad`$= 1$ (Appendix A).

Intuition

Why reverse, not forward? Our loss is one scalar but it has three leaves. Reverse mode seeds the *single* output and fans gradient out to *all* leaves in one pass — cost $\approx$ one extra forward pass, regardless of how many parameters there are. Forward mode would seed one *input* at a time and need a separate pass per leaf. A real network has the same shape: one scalar loss, millions of parameters. Reverse-mode (backprop) gets all million gradients for the price of roughly one forward pass; that asymmetry is the whole reason deep learning is affordable.

5 Step 4 — The dead-ReLU branch: $c < 0$ kills its gradient

The ReLU is the only nonlinear node, and its derivative is a switch. Re-run the graph with $c = -4.0$ instead of $+4.0$, leaving a, b alone. Forward:

$$r = \mathrm{relu}(-4.0) = \max(0, -4.0) = 0.0, \qquad f = u + r = -6.0 + 0.0 = -6.0.$$

Now the local derivative flips off because the input is negative:

$$\left.\frac{\partial r}{\partial c}\right|_{c=-4} = 0 \quad \Longrightarrow \quad \frac{\partial L}{\partial c} = \bar{r} \cdot 0 = 1 \cdot 0 = \boxed{0}.$$

The gradient to c is exactly zero — the backward signal hits a closed gate and stops. The multiply branch is untouched: $\partial L/\partial a$ and $\partial L/\partial b$ are still -3 and 2. PyTorch returns `c.grad`$= 0$ here (Appendix A).

Watch out

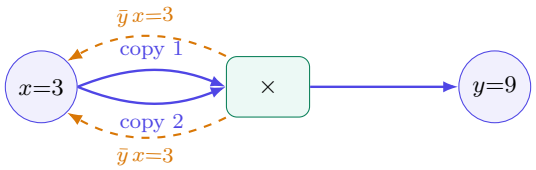
A unit whose ReLU input is negative for *every* training example receives zero gradient forever — a "dead ReLU." It cannot learn its way back to life because the gate that would let gradient through is the same gate that is stuck closed. This is a real failure mode (often from a too-large learning rate or bad initialisation) and the motivation for variants like LeakyReLU, whose negative-side slope keeps a small nonzero $\partial r/\partial c$.

6 Step 5 — Fan-out: when one tensor feeds two edges, gradients add

So far each leaf fed exactly one node. What if a tensor is used *twice*? Consider the self-product

$$y = x \cdot x, \qquad x = 3.0 \;\Rightarrow\; y = 9.0.$$

In the graph, x is a single leaf with *two* outgoing edges into the multiply node: one as the left factor, one as the right factor.



Treat the two uses as distinct inputs x_1, x_2 of the product $y = x_1 x_2$. The local derivatives are $\partial y/\partial x_1 = x_2 = 3$ and $\partial y/\partial x_2 = x_1 = 3$. Seeding $\bar{y} = 1$, each edge delivers a gradient of 3 back to x. Because x is really *one* tensor, those two contributions must be *summed* at the fork:

$$\frac{\partial y}{\partial x} = \underbrace{\bar{y}\,\frac{\partial y}{\partial x_1}}_{=3} + \underbrace{\bar{y}\,\frac{\partial y}{\partial x_2}}_{=3} = 3 + 3 = \boxed{6} \;=\; 2x\big|_{x=3}.$$

The hand-rule "$\frac{d}{dx}x^2 = 2x$" and the graph's "add the two edge gradients" are the same statement. PyTorch returns `x.grad`$= 6$ (Appendix A).

Intuition

The summation-at-a-fork rule is the multivariable chain rule: if L depends on x through several paths, $\partial L/\partial x$ is the *sum* over paths of the product of local derivatives along each path. A fan-out node is just a place where two paths rejoin, so their gradients add. This is also why `.backward()` *accumulates* into `.grad` rather than overwriting — accumulation is exactly how it merges the paths.

7 Step 6 — Why `.grad` accumulates and must be zeroed

That accumulation is convenient inside one backward pass but dangerous *across* steps. PyTorch *adds* new gradients into the existing `.grad` buffer. Call `backward()` twice on $y = x \cdot x$ at $x = 3$ without clearing in between:

$$\text{after 1st backward: } x.\text{grad} = 6, \qquad \text{after 2nd backward: } x.\text{grad} = 6 + 6 = 12.$$

The second value is wrong as a gradient — it is two gradients stacked. Zeroing the buffer first restores the correct number:

$$x.\text{grad.zero\_}(); \quad \text{backward}() \;\Rightarrow\; x.\text{grad} = 6.$$

Watch out

In every training loop you must clear gradients before each backward pass — typically `optimizer.zero_grad()` (or `model.zero_grad()`) right before `loss.backward()`. Forget it and your gradients silently sum across iterations, so each step uses a stale, ballooning gradient and training diverges or stalls. The first thing to check when "my loss explodes after step one" is a missing `zero_grad`.

When this is the right choice

Accumulation is a feature, not just a footgun: *deliberately* skipping `zero_grad` across several mini-batches before one `optimizer.step()` is exactly "gradient accumulation," the standard trick for simulating a large batch on limited memory. The rule is simply: zero once per *effective* batch, not once per forward pass.

8 The whole pass, at a glance

#	Quantity	Formula	Value	Verified
1	u (forward)	$a \cdot b$	-6.0	✓
2	r (forward)	$\max(0, c)$	4.0	✓
3	$f = L$ (forward)	$u + r$	-2.0	✓
4	$\bar{f}$ (seed)	$\partial L/\partial f$	1	✓
5	$\bar{u},\ \bar{r}$	$\bar{f} \cdot 1$	$1,\ 1$	✓
6	$\partial L/\partial a$	$\bar{u} \cdot b$	-3	✓
7	$\partial L/\partial b$	$\bar{u} \cdot a$	2	✓
8	$\partial L/\partial c$	$\bar{r} \cdot [c > 0]$	1	✓
9	dead ReLU (c=-4)	$\bar{r} \cdot [c > 0]$	0	✓
10	fan-out y=x^2	$\bar{y}x + \bar{y}x = 2x$	6	✓

Ten numbers, one forward sweep and one backward sweep. Every leaf gradient came from multiplying stored local derivatives along graph edges and summing where paths rejoin.

9 Equation cheat-sheet

Add node	$f = u + r: \quad \partial f/\partial u = 1,\ \partial f/\partial r = 1$
Multiply node	$u = ab: \quad \partial u/\partial a = b,\ \partial u/\partial b = a$
ReLU node	$r = \max(0, c): \quad \partial r/\partial c = [c > 0]$
Adjoint / seed	$\bar{v} = \partial L/\partial v, \qquad \bar{f} = 1$
Backward (one edge)	$(\bar{\text{in}}) \mathrel{+}= (\bar{\text{out}}) \cdot \partial(\text{out})/\partial(\text{in})$
Fan-out (multi-path)	$\partial L/\partial x = \sum_{\text{paths}} \prod_{\text{edges}} (\text{local})$
Reverse-mode cost	all leaf grads in ≈ 1 extra forward pass
Zero between steps	`optimizer.zero_grad()` before each `loss.backward()`

Appendix A Reference implementation

Every number above is reproducible from the following program. The hand-derived gradients are printed alongside `torch.autograd`'s, which match to the digit.

```
import torch

# ---- the tiny graph: f = (a*b) + relu(c),  L = f ----
a = torch.tensor(2.0,  requires_grad=True)
b = torch.tensor(-3.0, requires_grad=True)
c = torch.tensor(4.0,  requires_grad=True)

u = a * b                 # forward: u = -6.0
r = torch.relu(c)         # forward: r =  4.0
f = u + r                 # forward: f = -2.0  (= L)
print("forward  u, r, f =", u.item(), r.item(), f.item())

f.backward()              # one reverse-mode sweep -> all leaf grads
print("dL/da, dL/db, dL/dc =", a.grad.item(), b.grad.item(), c.grad.item())
# -> -3.0 2.0 1.0   (matches the by-hand boxes)

# ---- dead-ReLU branch: c < 0 kills its gradient ----
c2 = torch.tensor(-4.0, requires_grad=True)
f2 = (a.detach() * b.detach()) + torch.relu(c2)   # f2 = -6.0
f2.backward()
print("dead relu, dL/dc =", c2.grad.item())       # -> 0.0

# ---- fan-out: y = x*x, gradients ADD at the fork ----
x = torch.tensor(3.0, requires_grad=True)
y = x * x                 # y = 9.0
y.backward()
print("fan-out dy/dx =", x.grad.item())            # -> 6.0  ( = 2x )

# ---- .grad ACCUMULATES; must be zeroed between steps ----
p = torch.tensor(3.0, requires_grad=True)
(p * p).backward(); print("after 1 backward:", p.grad.item())   # 6.0
(p * p).backward(); print("after 2 backward:", p.grad.item())   # 12.0  (stacked!)
p.grad.zero_()
(p * p).backward(); print("after zero_ + backward:", p.grad.item())  # 6.0
```

— end of document —